Exploring Generative AI and AI Fundamentals

Core Concepts, Model Training, Prompt Engineering, APIs, and a Practical AI Application

Assignment: Exploring Generative AI and AI Fundamentals

Prepared on: 29 October 2026

Research report with practical AI application and references

Abstract

Generative artificial intelligence (GenAI) refers to AI systems that learn patterns from data and create new content, including text, images, audio, video, and code. This report introduces foundational AI concepts, explains how generative models are trained and evaluated, and discusses prompt engineering and API integration. It also presents a simple practical application: a Python-based study assistant that uses a generative model to answer questions and create revision material. The report concludes with limitations and responsible-use considerations, including privacy, bias, hallucinations, security, and human oversight.

1. Introduction to Artificial Intelligence

Artificial intelligence is a field of computing focused on systems that perform tasks commonly associated with human intelligence, such as recognising patterns, understanding language, making predictions, planning, and solving problems. Machine learning (ML) is a subset of AI in which systems learn patterns from data rather than relying only on explicitly written rules. Deep learning uses neural networks with multiple layers to learn complex representations.

Generative AI differs from many traditional predictive systems. A predictive model may classify an email as spam or estimate a value, while a generative model creates an output such as a paragraph, image, or program. Generative models can support creativity and productivity, but their outputs still require verification.

2. Foundational Models and Generative AI

A foundation model is trained on broad data and can be adapted to multiple tasks. Large language models (LLMs) are foundation models designed primarily for language tasks. They commonly process text as tokens and generate a sequence by predicting likely next tokens. Other generative model families include diffusion models for images and video, generative adversarial networks (GANs), and variational autoencoders (VAEs).

Common applications include summarisation, translation, conversational assistants, code generation, content creation, document search, and tutoring. Some models are multimodal, meaning they can work with more than one type of input or output, such as text and images. Model choice depends on the task, quality requirements, latency, cost, privacy, and supported modalities.

3. Model Training Lifecycle

Data preparation: training data is collected and processed under relevant quality, privacy, licensing, and consent requirements. It may be filtered, deduplicated, labelled, and tokenised.

Pre-training: the model learns general patterns from large datasets. For many language models, training includes predicting the next token. The model’s parameters are updated to reduce prediction error across many examples.

Fine-tuning: a pre-trained model can be adapted to a specialised task using curated examples.
Parameter-efficient methods may reduce the compute required for adaptation.

Instruction tuning and alignment: instruction examples help a model follow user requests. Preference optimisation can further encourage desirable responses, but does not guarantee truthfulness or eliminate bias.

Evaluation: testing measures task accuracy, factuality, robustness, bias, privacy leakage, security, response time, and cost. Evaluations should reflect the real context in which the model will be used.

Deployment and monitoring: after launch, teams monitor failures, feedback, changing data, model versions, and operational cost. High-impact systems need defined escalation paths and qualified human review.

4. Prompt Engineering

Prompt engineering is the practice of writing instructions and providing context so that a model can respond more reliably. A useful prompt usually states the role or task, gives necessary context, defines constraints, specifies the desired output format, and explains how uncertainty should be handled.

Example prompt: “Act as a university study assistant. Explain the difference between supervised and unsupervised learning in simple language. Give one engineering example of each, compare them in a table, and finish with three practice questions. If a detail is uncertain, say so instead of inventing facts.”

Useful techniques include providing examples, breaking a complex task into smaller steps, specifying a target audience, asking for structured output, and grounding the answer in supplied documents. Prompts should not include passwords, private keys, or unnecessary personal data. Prompt engineering improves task clarity, but it cannot fully compensate for weak models or unreliable sources.

5. Using APIs in AI Applications

An application programming interface (API) allows one software system to request services from another. A hosted AI API lets a developer send a prompt to a model provider and receive a generated response. A typical flow is: user enters a question → application validates the input → backend calls the model API → response is checked and returned to the user.

For a real implementation, the developer should select a provider and model, read its current documentation, create an API key, and store the key in a server-side environment variable. The key must never be placed in public frontend code or committed to a public repository. Applications should handle network failures, rate limits, timeouts, invalid inputs, and usage costs. User data should be minimised and processed according to the provider’s privacy terms and applicable rules.

6. Practical Application: AI Study Assistant

Objective: build a simple assistant that helps students revise a topic by generating explanations, examples, and practice questions.

Features:
• Topic explanation: produces a concise explanation at the requested academic level.
• Example generation: gives a practical example related to engineering or computing.
• Revision quiz: creates a small set of practice questions.
• Summary mode: converts a long set of notes into key points.
• Safe fallback: asks the student to verify important facts and does not claim that generated content is guaranteed correct.

Suggested architecture: a simple web interface or command-line interface collects the topic and preferred task. A backend validates the request and calls a selected generative model API. The backend returns the answer and optionally records non-sensitive usage statistics. For a more reliable study assistant, course notes can be indexed and relevant passages retrieved before generation; this is called retrieval-augmented generation (RAG).

Example Python structure (provider-neutral pseudocode):
```python
import os

API_KEY = os.environ.get("AI_API_KEY")
if not API_KEY:
raise RuntimeError("Set AI_API_KEY in your environment.")

def build_prompt(topic, task):
return (
```

```
"You are a university study assistant. "
"Use clear language and an engineering example. "
"If evidence is insufficient, say so.\n"
f"Topic: {topic}\nTask: {task}"
)

# Use your chosen provider's official SDK here:
# response = client.generate(build_prompt(topic, task))
# print(response.text)
```

The commented API call is intentionally provider-neutral. To make the project runnable, a developer must select a provider, install its current SDK, configure the API key securely, and follow that provider's current API documentation.

7. Evaluation Strategy

A simple evaluation plan should include a test set of representative topics and questions. Review whether explanations are correct, whether examples fit the topic, whether the quiz matches the requested difficulty, and whether the assistant admits when it lacks enough evidence. Measure response time, cost per request, and failure rate. Include edge cases such as empty input, very long prompts, ambiguous questions, and questions outside the available course material.

Before deployment, compare the AI assistant's answers with trusted course notes or a subject expert's reference answers. Ask a small group of users to test the interface and report confusing or inaccurate outputs. Keep a record of recurring failures and re-test after changes to the model, prompt, or retrieval system.

8. Risks and Responsible Use

Hallucinations: generative models can produce plausible but false facts, citations, or code. Verify important claims and use trusted sources.

Bias and fairness: model outputs may reflect bias in training data. Test representative examples and provide a way to report issues.

Privacy: avoid sending confidential information or unnecessary personal data to a model. Apply data minimisation, access controls, and suitable retention policies.

Security: protect API keys on the backend, validate inputs, apply rate limits, and guard against prompt injection if the application retrieves external documents.

Copyright and attribution: check the rights and terms associated with training data, generated material, and third-party assets. Follow institutional rules for acknowledging AI assistance.

Human oversight: generated content should support learning and professional judgement, not replace them. Students should verify explanations, understand the material, and comply with academic integrity requirements.

9. Conclusion

Generative AI builds on machine learning and deep learning to produce useful content across text, images, audio, video, and code. Understanding foundation models, training, prompt engineering, API integration, and evaluation helps developers create more dependable applications. A study assistant is a practical starter project because it has a clear purpose and can be evaluated against course materials. However, reliable deployment requires secure API-key handling, privacy protection, robust testing, source verification, and human oversight. Generative AI is most useful when its strengths are matched to a well-defined task and its limitations are made explicit.

References

[1] Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press. https://www.deeplearningbook.org/

[2] Vaswani, A., et al. (2017). Attention Is All You Need. Advances in Neural Information Processing Systems. https://arxiv.org/abs/1706.03762

[3] Bommasani, R., et al. (2021). On the Opportunities and Risks of Foundation Models. https://arxiv.org/abs/2108.07258

[4] National Institute of Standards and Technology. (2024). Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile (NIST AI 600-1). https://doi.org/10.6028/NIST.AI.600-1

This report is for educational purposes. Choose a model provider and verify current API documentation, pricing, privacy terms, and academic rules before implementation.